MODUL PRAKTIKUM

PRAKTIKUM DASAR PEMROGRAMAN

Modul 2

Input, Output, Variabel, dan Tipe Data

Disusun oleh:

Dr. Desta Sandya Prasvita, S.Komp., M.Kom.

Tim *Teaching*

PROGRAM STUDI S1 INFORMATIKA

UPN VETERAN JAKARTA

2026

Modul ini melanjutkan Modul 1 dan mendukung bahan kajian data, variabel, operator dasar, serta implementasi algoritma sederhana.

I. Pendahuluan Modul

1. Identitas Modul

Komponen	Keterangan
Mata Kuliah	Praktikum Dasar Pemrograman
Program Studi	S1 Informatika
Semester	1
Modul	2
Judul	Input, Output, Variabel, dan Tipe Data
Bahasa Pemrograman	C
Alokasi Waktu	1 × 170 menit
Metode	Demonstrasi, latihan terbimbing, eksperimen, debugging, dan latihan mandiri
Prasyarat	Modul 1: Pengenalan Lingkungan Pemrograman dan Struktur Program C

3. Indikator Keberhasilan

Mahasiswa dinyatakan berhasil apabila mampu:

1. Mahasiswa mampu mendeklarasikan dan menginisialisasi variabel dengan tipe data yang sesuai.
2. Mahasiswa mampu menggunakan printf() dan scanf() dengan format specifier yang tepat.
3. Mahasiswa mampu memilih tipe data dasar int, float, double, dan char sesuai karakteristik data.
4. Mahasiswa mampu menerapkan konsep Input-Process-Output (IPO) dalam program sederhana.
5. Mahasiswa mampu menemukan dan memperbaiki kesalahan dasar pada penggunaan variabel, tipe data, dan format specifier.

II. Persiapan Praktikum

Persiapan Folder Praktikum

Buat struktur folder berikut:

```
Praktikum_Dasar_Pemrograman/
└── Modul_02/
    ├── latihan_01.c
    ├── latihan_02.c
    ├── latihan_03.c
    ├── eksperimen.c
    ├── debugging.c
    └── tugas_modul_02.c
```

Gunakan nama berkas dengan ketentuan:

- menggunakan huruf kecil;
- tidak menggunakan spasi;
- menggunakan garis bawah _ sebagai pemisah;
- menggunakan ekstensi .c.

III. Konsep Dasar

1. Data

Data adalah nilai atau informasi yang diolah oleh program.

Contoh data:

- nama mahasiswa;
- umur;
- jumlah mata kuliah;
- nilai ujian;
- tinggi badan;
- status kelulusan.

Data dapat memiliki bentuk yang berbeda sehingga perlu direpresentasikan dengan tipe data yang sesuai.

2. Variabel

Variabel adalah tempat untuk menyimpan nilai selama program dijalankan.

Setiap variabel memiliki:

- nama;
- tipe data;
- nilai;
- lokasi penyimpanan di memori.

Contoh deklarasi:

```
int umur;
```

Pada pernyataan tersebut:

- int adalah tipe data;
- umur adalah nama variabel.

3. Deklarasi Variabel

Deklarasi digunakan untuk memberi tahu compiler bahwa program akan menggunakan variabel tertentu.

Bentuk umum:

```
tipe_data nama_variabel;
```

Contoh:

```
int jumlah_mahasiswa;
float nilai_rata_rata;
double jarak;
char huruf_mutu;
```

Beberapa variabel dengan tipe yang sama dapat dideklarasikan bersama:

```
int panjang, lebar, luas;
```

Namun, untuk mahasiswa pemula, deklarasi terpisah sering lebih mudah dibaca:

```
int panjang;
int lebar;
int luas;
```

4. Inisialisasi Variabel

Inisialisasi adalah pemberian nilai awal kepada variabel.

Contoh:

```
int semester = 1;
float nilai = 85.5f;
char huruf_mutu = 'A';
```

Deklarasi tanpa inisialisasi:

```
int jumlah;
```

Deklarasi sekaligus inisialisasi:

```
int jumlah = 10;
```

Variabel lokal sebaiknya tidak digunakan sebelum memperoleh nilai yang valid.

Contoh yang tidak disarankan:

```
int jumlah;
printf("%d\n", jumlah);
```

Nilai jumlah belum ditentukan sehingga hasilnya tidak dapat diandalkan.

5. Aturan Penamaan Variabel

Nama variabel dalam bahasa C harus memenuhi aturan berikut:

- dapat menggunakan huruf, angka, dan garis bawah;
- tidak boleh dimulai dengan angka;
- tidak boleh mengandung spasi;
- tidak boleh menggunakan karakter khusus seperti @, #, atau -;
- tidak boleh sama dengan kata kunci bahasa C;
- bersifat case-sensitive.

Contoh nama yang valid:

```
nilai
nilai_akhir
jumlahMahasiswa
semester_1
```

Contoh nama yang tidak valid:

```
1nilai
nilai akhir
jumlah-mahasiswa
int
```

Nama variabel yang disarankan

Gunakan nama yang menggambarkan isi variabel.

Kurang informatif:

```
int x;
int y;
int z;
```

Lebih informatif:

```
int panjang;
int lebar;
int luas;
```

Untuk perhitungan koordinat atau rumus matematika tertentu, nama singkat seperti x dan y tetap dapat digunakan apabila konteksnya jelas.

6. Tipe Data Dasar

a. int

Tipe int digunakan untuk menyimpan bilangan bulat.

Contoh:

```
int umur = 18;
int jumlah_mahasiswa = 35;
int semester = 1;
```

Contoh nilai:

```
-10
0
25
1000
```

b. float

Tipe float digunakan untuk menyimpan bilangan riil atau pecahan dengan presisi terbatas.

Contoh:

```
float tinggi_badan = 170.5f;
float nilai_rata_rata = 82.75f;
```

Huruf f menunjukkan bahwa literal tersebut diperlakukan sebagai float.

```
82.75f
```

c. double

Tipe double digunakan untuk menyimpan bilangan riil dengan ketelitian yang umumnya lebih tinggi daripada float.

Contoh:

```
double jarak = 1250.756;
double harga = 150000.50;
```

Untuk sebagian besar perhitungan yang membutuhkan bilangan pecahan, double sering menjadi pilihan yang lebih aman.

d. char

Tipe char digunakan untuk menyimpan satu karakter.

Contoh:

```
char huruf_mutu = 'A';
char jenis_kelamin = 'L';
char pilihan = 'Y';
```

Nilai char ditulis menggunakan tanda petik tunggal:

```
'A'
```

Tanda petik ganda digunakan untuk teks atau string:

```
"Algoritma"
```

Pada tahap ini mahasiswa perlu membedakan:

```
'A'
```

dengan:

```
"A"
```

Yang pertama adalah satu karakter, sedangkan yang kedua adalah rangkaian karakter.

e. Ukuran Tipe Data

Ukuran tipe data dapat berbeda bergantung pada compiler, arsitektur komputer, dan sistem yang digunakan.

Gunakan operator sizeof untuk melihat ukuran aktual.

```
#include <stdio.h>

int main(void)
{
    printf("Ukuran int    : %zu byte\n", sizeof(int));
    printf("Ukuran float  : %zu byte\n", sizeof(float));
    printf("Ukuran double : %zu byte\n", sizeof(double));
    printf("Ukuran char   : %zu byte\n", sizeof(char));

    return 0;
}
```

Jangan mengasumsikan seluruh compiler selalu memberikan ukuran yang sama untuk setiap tipe data.

7. Konstanta

Konstanta adalah nilai yang tidak seharusnya berubah selama program dijalankan.

Menggunakan const

```
const double PHI = 3.141592653589793;
```

Contoh penggunaan:

```
double jari_jari = 7.0;
double luas = PHI * jari_jari * jari_jari;
```

Konstanta biasanya diberi nama menggunakan huruf kapital agar mudah dibedakan dari variabel.

Contoh:

```
const int JUMLAH_BULAN = 12;
const double PAJAK = 0.11;
```

8. Output Menggunakan printf()

Fungsi printf() digunakan untuk menampilkan teks dan nilai variabel.

Menampilkan teks

```
printf("Praktikum Dasar Pemrograman\n");
```

Menampilkan bilangan bulat

```
int umur = 18;
printf("Umur: %d tahun\n", umur);
```

Menampilkan bilangan pecahan

```
float nilai = 85.5f;
printf("Nilai: %.2f\n", nilai);
```

Menampilkan karakter

```
char huruf_mutu = 'A';
printf("Huruf mutu: %c\n", huruf_mutu);
```

9. Format Specifier

Format specifier memberi tahu printf() atau scanf() tentang tipe data yang digunakan.

Format untuk printf()

Tipe data	Format	Contoh
int	%d	printf("%d", jumlah);
float	%f	printf("%f", nilai);
double	%f	printf("%f", jarak);
char	%c	printf("%c", huruf);
ukuran dari sizeof	%zu	printf("%zu", sizeof(int));

Pada printf(), nilai float secara otomatis dipromosikan menjadi double. Oleh karena itu, %f digunakan untuk menampilkan float maupun double.

Format untuk scanf()

Tipe data	Format	Contoh
int	%d	scanf("%d", &jumlah);
float	%f	scanf("%f", &nilai);
double	%lf	scanf("%lf", &jarak);
char	%c	scanf(" %c", &huruf);

Perhatikan bahwa pada scanf():

- float menggunakan %f;
- double menggunakan %lf.

Kesalahan membedakan keduanya dapat menyebabkan nilai input tidak tersimpan dengan benar.

10. Mengatur Tampilan Bilangan Pecahan

Jumlah angka di belakang koma dapat diatur.

```
double nilai = 85.7564;

printf("%.1f\n", nilai);
printf("%.2f\n", nilai);
printf("%.3f\n", nilai);
```

Keluaran:

```
85.8
85.76
85.756
```

Bentuk:

```
%.2f
```

berarti menampilkan dua angka di belakang koma.

Lebar tampilan juga dapat diatur:

```
printf("%10.2f\n", nilai);
```

Angka 10 merupakan lebar minimum bidang keluaran.

11. Input Menggunakan scanf()

Fungsi scanf() digunakan untuk menerima input dari pengguna.

Contoh:

```
#include <stdio.h>

int main(void)
{
    int umur;

    printf("Masukkan umur: ");
    scanf("%d", &umur);

    printf("Umur Anda adalah %d tahun.\n", umur);

    return 0;
}
```

Mengapa menggunakan &?

Pada pemakaian dasar scanf(), simbol & digunakan untuk memberikan alamat variabel tempat hasil input akan disimpan.

```
scanf("%d", &umur);
```

Konsep alamat memori dan pointer akan dibahas lebih mendalam pada modul berikutnya.

Untuk saat ini, pahami aturan:

Pada input numerik dan karakter menggunakan scanf(), nama variabel umumnya didahului dengan &.

12. Memeriksa Keberhasilan Input

scanf() mengembalikan jumlah data yang berhasil dibaca.

Contoh:

```
#include <stdio.h>

int main(void)
{
    int umur;
    int hasil_input;

    printf("Masukkan umur: ");
    hasil_input = scanf("%d", &umur);

    if (hasil_input == 1)
    {
        printf("Input berhasil dibaca.\n");
        printf("Umur: %d tahun\n", umur);
    }
    else
    {
        printf("Input tidak valid.\n");
    }
    return 0;
}
```

Struktur if akan dibahas secara khusus pada modul kondisional. Pada Modul 2, contoh tersebut digunakan untuk menunjukkan bahwa proses input dapat diperiksa.

Untuk latihan dasar, mahasiswa dapat menggunakan pola:

```
if (scanf("%d", &umur) != 1)
{
    printf("Input tidak valid.\n");
    return 1;
}
```

13. Input Karakter

Contoh:

```
#include <stdio.h>
int main(void)
{
    char huruf_mutu;

    printf("Masukkan huruf mutu: ");
    scanf(" %c", &huruf_mutu);

    printf("Huruf mutu: %c\n", huruf_mutu);
```

```
    return 0;
}
```

Perhatikan adanya spasi sebelum %c:

```
" %c"
```

Spasi tersebut membantu scanf() melewati karakter whitespace seperti baris baru yang masih tersisa pada input sebelumnya.

14. Konsep Input–Process–Output

Program sederhana dapat dianalisis menggunakan tiga bagian.

Input

Data yang dimasukkan ke dalam program.

Process

Operasi yang dilakukan terhadap data.

Output

Hasil yang ditampilkan oleh program.

Contoh perhitungan luas persegi panjang:

```
Input   : panjang dan lebar
Process : luas = panjang × lebar
Output  : luas
```

Implementasi:

```
#include <stdio.h>

int main(void)
{
    double panjang;
    double lebar;
    double luas;

    printf("Masukkan panjang: ");
    scanf("%lf", &panjang);

    printf("Masukkan lebar: ");
    scanf("%lf", &lebar);

    luas = panjang * lebar;

    printf("Luas persegi panjang: %.2f\n", luas);

    return 0;
}
```

IV. Praktikum Terbimbing

1. Praktikum Terbimbing 1 — Membaca dan Menampilkan Identitas Numerik

Buat berkas: latihan_01.c. Tuliskan program berikut:

```
#include <stdio.h>

int main(void)
{
    int umur;
    int semester;
    float tinggi_badan;

    printf("Masukkan umur: ");
    scanf("%d", &umur);

    printf("Masukkan semester: ");
    scanf("%d", &semester);

    printf("Masukkan tinggi badan dalam cm: ");
    scanf("%f", &tinggi_badan);

    printf("\nDATA MAHASISWA\n");
    printf("Umur          : %d tahun\n", umur);
    printf("Semester      : %d\n", semester);
    printf("Tinggi badan  : %.1f cm\n", tinggi_badan);

    return 0;
}
```

Contoh eksekusi

```
Masukkan umur: 18
Masukkan semester: 1
Masukkan tinggi badan dalam cm: 170.5

DATA MAHASISWA
Umur          : 18 tahun
Semester      : 1
Tinggi badan  : 170.5 cm
```

Pertanyaan analisis

1. Mengapa umur menggunakan tipe int?
2. Mengapa tinggi_badan menggunakan float?
3. Apa fungsi &umur pada scanf()?
4. Apa arti %.1f?
5. Apa yang terjadi apabila pengguna mengetik teks saat program meminta umur?
6. Apa yang terjadi apabila %f untuk tinggi_badan diganti menjadi %d?
7. Apakah tinggi badan dapat menggunakan double? Jelaskan.

2. Praktikum Terbimbing 2 — Menghitung Luas dan Keliling Persegi Panjang

Buat berkas: latihan_02.c. Gunakan program berikut:

```
#include <stdio.h>
int main(void)
{
    double panjang;
    double lebar;
    double luas;
    double keliling;

    printf("PROGRAM PERSEGI PANJANG\n");
    printf("=======================\n");

    printf("Masukkan panjang: ");
    scanf("%lf", &panjang);

    printf("Masukkan lebar: ");
    scanf("%lf", &lebar);

    luas = panjang * lebar;
    keliling = 2.0 * (panjang + lebar);

    printf("\nHASIL PERHITUNGAN\n");
    printf("Panjang   : %.2f\n", panjang);
    printf("Lebar     : %.2f\n", lebar);
    printf("Luas      : %.2f\n", luas);
    printf("Keliling  : %.2f\n", keliling);
    return 0;
}
```

Analisis IPO

Bagian	Data
Input	panjang dan lebar
Process	luas = panjang * lebar
Process	keliling = 2 * (panjang + lebar)
Output	luas dan keliling

Contoh input: Panjang: 10.5, Lebar: 4

Keluaran yang diharapkan:

```
Panjang   : 10.50
Lebar     : 4.00
Luas      : 42.00
Keliling  : 29.00
```

Pertanyaan analisis

1. Mengapa variabel menggunakan double?
2. Mengapa input double menggunakan %lf?
3. Mengapa output double menggunakan %f, bukan %lf?
4. Apa kegunaan tanda kurung pada rumus keliling?
5. Bagaimana hasilnya apabila panjang bernilai 0?

6. Apakah nilai negatif masuk akal untuk panjang atau lebar?
7. Validasi seperti apa yang seharusnya dilakukan?

3. Praktikum Terbimbing 3 — Konversi Suhu

Buat berkas: latihan_03.c. Program menerima suhu Celsius dan mengubahnya menjadi Fahrenheit.

Rumus:

```
Fahrenheit = (9 ÷ 5 × Celsius) + 32
```

Implementasi:

```
#include <stdio.h>

int main(void)
{
    double celsius;
    double fahrenheit;

    printf("Masukkan suhu Celsius: ");
    scanf("%lf", &celsius);

    fahrenheit = (9.0 / 5.0) * celsius + 32.0;

    printf("%.2f derajat Celsius = %.2f derajat Fahrenheit\n",
           celsius,
           fahrenheit);

    return 0;
}
```

Perhatikan penggunaan:

```
9.0 / 5.0
```

Bukan:

```
9 / 5
```

Dalam bahasa C, pembagian dua bilangan bulat menghasilkan pembagian bilangan bulat.

```
9 / 5
```

menghasilkan:

```
1
```

sedangkan:

```
9.0 / 5.0
```

menghasilkan:

```
1.8
```

Perbedaan ini akan dibahas lebih lanjut pada Modul 3 tentang operator dan ekspresi.

V. Eksperimen dan Tracing

1. Eksperimen

Buat berkas: eksperimen.c. Gunakan program awal berikut:

```
#include <stdio.h>

int main(void)
{
    int jumlah = 25;
    float nilai_float = 85.756f;
    double nilai_double = 85.756;
    char huruf = 'A';

    printf("Jumlah       : %d\n", jumlah);
    printf("Float        : %f\n", nilai_float);
    printf("Double       : %f\n", nilai_double);
    printf("Karakter     : %c\n", huruf);

    return 0;
}
```

Lakukan eksperimen satu per satu.

No.	Eksperimen	Prediksi	Hasil
1	Ganti %d untuk jumlah menjadi %f		
2	Ganti %.2f menjadi %.4f		
3	Ganti nilai huruf dari 'A' menjadi "A"		
4	Hapus huruf f dari 85.756f		
5	Tampilkan ukuran setiap tipe menggunakan sizeof		
6	Gunakan nama Nilai dan nilai sebagai dua variabel		
7	Gunakan variabel tanpa inisialisasi		
8	Ganti 9.0 / 5.0 menjadi 9 / 5 pada program suhu		

Untuk setiap eksperimen:

- tuliskan prediksi;
- kompilasi program;
- catat peringatan atau error;
- jalankan program apabila kompilasi berhasil;
- bandingkan hasil dengan prediksi;
- kembalikan source code ke kondisi awal.

VI. Debugging Challenge

1. Debugging Challenge

Program berikut memiliki sejumlah kesalahan.

```
#include <stdio.h>

int main(void)
{
    int umur
    float tinggi_badan;
    char huruf_mutu;

    printf("Masukkan umur: ");
    scanf("%f", umur);

    printf("Masukkan tinggi badan: ");
    scanf("%lf", &tinggi_badan);

    printf("Masukkan huruf mutu: ");
    scanf("%c", huruf_mutu);

    printf("\nUmur: %d tahun\n", umur);
    printf("Tinggi badan: %d cm\n", tinggi_badan);
    printf("Huruf mutu: %s\n", huruf_mutu);

    return 0;
}
```

Tugas

1. Salin program ke berkas debugging.c.
2. Kompilasi program tanpa langsung memperbaikinya.
3. Catat pesan error atau warning.
4. Identifikasi semua kesalahan.
5. Perbaiki satu kesalahan pada satu waktu.
6. Kompilasi ulang setelah setiap perbaikan.
7. Jalankan program dengan data uji.
8. Pastikan hasil akhirnya benar.

Gunakan tabel berikut:

No.	Bagian yang Salah	Penyebab	Perbaikan
1			
2			
3			
4			
5			
6			

7			

Petunjuk

Periksa:

- titik koma;
- format specifier;
- penggunaan &;
- perbedaan float dan double;
- format karakter;
- spasi sebelum %c.

VII. Latihan Mandiri

1. Melengkapi Program

Lengkapi program berikut:

```
#include <stdio.h>

int main(void)
{
    int jumlah_sks;
    float nilai_rata_rata;
    char kelas;

    printf("Masukkan jumlah SKS: ");
    scanf("____", ____________);

    printf("Masukkan nilai rata-rata: ");
    scanf("____", ____________);

    printf("Masukkan kelas: ");
    scanf("____", ____________);

    printf("\nJumlah SKS       : ____\n", jumlah_sks);
    printf("Nilai rata-rata  : ____\n", nilai_rata_rata);
    printf("Kelas            : ____\n", kelas);

    return 0;
}
```

2. Menghitung Usia

Buat program yang menerima tahun lahir dan tahun sekarang, kemudian menghitung perkiraan usia.

Input:

- tahun lahir;
- tahun sekarang.

Process:

```
usia = tahun sekarang − tahun lahir
```

Output:

- tahun lahir;
- tahun sekarang;
- perkiraan usia.

Contoh:

```
Masukkan tahun lahir: 2007
Masukkan tahun sekarang: 2026

Perkiraan usia: 19 tahun
```

Pada modul ini, program belum wajib memvalidasi apakah tahun lahir lebih besar daripada tahun sekarang. Validasi tersebut akan ditambahkan setelah mahasiswa mempelajari seleksi.

3. Menghitung Total Belanja

Buat program yang menerima:

- nama kode barang berupa satu karakter;
- harga satuan;
- jumlah barang.

Program menghitung:

```
total = harga satuan × jumlah barang
```

Contoh interaksi:

```
Masukkan kode barang: A
Masukkan harga satuan: 12500
Masukkan jumlah barang: 4

RINCIAN TRANSAKSI
Kode barang  : A
Harga satuan : Rp12500.00
Jumlah       : 4
Total        : Rp50000.00
```

Gunakan:

- char untuk kode barang;
- double untuk harga;
- int untuk jumlah.

4. Perhitungan Waktu

Buat program yang menerima total waktu dalam satuan detik dan mengubahnya menjadi jam, menit, dan detik.

Contoh:

```
Masukkan total detik: 3725

3725 detik setara dengan:
1 jam
2 menit
5 detik
```

Petunjuk:

```
jam          = total detik ÷ 3600
sisa         = total detik % 3600
menit        = sisa ÷ 60
detik        = sisa % 60
```

Operator % akan dibahas lebih mendalam pada Modul 3.

5. Struk Pembelian Token Listrik

Sebuah minimarket ingin mencetak struk pembelian token listrik. Buat program C yang menerima daya (dalam kWh) dan harga per kWh, lalu mencetak struk dengan format presisi seperti pada Contoh Output.

Format Input

Dua baris angka bertipe double: daya (kWh) dan harga per kWh (rupiah).

Batasan

- $1 \leq$ daya ≤ 500 (dapat berupa bilangan pecahan dengan maksimal dua angka di belakang koma);
- $100 \leq$ harga per kWh ≤ 5000 (dapat berupa bilangan bulat maupun pecahan).

Contoh 1

Input:

```
10
1500
```

Output:

```
=======================================
         STRUK PEMBELIAN TOKEN
=======================================
Daya (kWh)    : 10.00
Harga per kWh : Rp1500.00
---------------------------------------
Total Bayar   : Rp15000.00
=======================================
Terima kasih telah menggunakan layanan ini.
```

Contoh 2

Input:

```
25.5
1200
```

Output:

```
=======================================
         STRUK PEMBELIAN TOKEN
=======================================
Daya (kWh)    : 25.50
Harga per kWh : Rp1200.00
---------------------------------------
Total Bayar   : Rp30600.00
=======================================
Terima kasih telah menggunakan layanan ini.
```

Contoh 3

Input:

```
7.25
2000
```

Output:

```
=======================================
         STRUK PEMBELIAN TOKEN
=======================================
Daya (kWh)    : 7.25
Harga per kWh : Rp2000.00
---------------------------------------
Total Bayar   : Rp14500.00
=======================================
Terima kasih telah menggunakan layanan ini.
```

Petunjuk Pendekatan

- gunakan variabel bertipe double untuk daya, harga, dan total;
- baca kedua nilai menggunakan scanf("%lf", ...);
- hitung total = daya * harga sebelum dicetak;
- gunakan %.2f untuk seluruh nilai uang dan daya agar selalu tampil dua angka di belakang koma;
- susun label (Daya, Harga per kWh, Total Bayar) dengan spasi manual agar tanda titik dua sejajar, seperti contoh pada Praktikum Terbimbing 1 dan 2;
- kedua baris pembatas (garis = dan garis -) harus memiliki jumlah karakter yang sama seperti pada Contoh Output.

VIII. Tugas Praktikum

1. Tugas Praktikum Modul 2 — Program Perhitungan Biaya Perjalanan

Buat program C untuk menghitung estimasi biaya bahan bakar suatu perjalanan.

Data input

Program menerima:

- jarak perjalanan dalam kilometer;
- konsumsi kendaraan dalam kilometer per liter;
- harga bahan bakar per liter;
- jumlah penumpang.

Proses perhitungan

```
kebutuhan bahan bakar = jarak ÷ konsumsi kendaraan
total biaya           = kebutuhan bahan bakar × harga per liter
biaya per orang       = total biaya ÷ jumlah penumpang
```

Contoh interaksi

```
=========================================
       ESTIMASI BIAYA PERJALANAN
=========================================
Masukkan jarak perjalanan (km)       : 150
Masukkan konsumsi kendaraan (km/l)   : 12.5
Masukkan harga bahan bakar per liter : 10000
Masukkan jumlah penumpang            : 4

HASIL PERHITUNGAN
-----------------------------------------
Jarak perjalanan       : 150.00 km
Kebutuhan bahan bakar  : 12.00 liter
Total biaya            : Rp120000.00
Biaya per orang        : Rp30000.00
```

```
========================================
```

Analisis IPO

Mahasiswa wajib melengkapi tabel berikut sebelum menulis program.

Komponen	Uraian
Input	
Process	
Output	

Ketentuan source code

Program wajib:

- menggunakan #include <stdio.h>;
- menggunakan int main(void);
- menggunakan tipe data yang sesuai;
- menggunakan nama variabel yang deskriptif;
- menerima seluruh data melalui scanf();
- menggunakan format specifier yang benar;
- menggunakan konstanta apabila diperlukan;
- menampilkan hasil dengan dua angka di belakang koma;
- menggunakan komentar identitas dan deskripsi program;
- memiliki indentasi yang konsisten;
- dapat dikompilasi tanpa error;
- diuji menggunakan minimal empat test case;
- dapat dijelaskan oleh mahasiswa.

Batasan Modul 2

Karena mahasiswa belum mempelajari seleksi secara formal, tugas belum mewajibkan penanganan kondisi seperti:

- konsumsi kendaraan bernilai nol;
- jumlah penumpang bernilai nol;
- jarak negatif;
- harga bahan bakar negatif.

Header komentar

Gunakan format berikut:

```
/*
Nama        :
NIM         :
Kelas       :
Modul       : 2
Deskripsi   : Program menghitung estimasi biaya perjalanan.
*/
```

Berkas yang dikumpulkan

- source code: tugas_modul_02_nim.c
- analisis input–process–output;
- tangkapan layar proses kompilasi;
- tangkapan layar program saat dijalankan;
- tabel pengujian;

- penjelasan pemilihan tipe data;
- pernyataan penggunaan AI.

Contoh:

```
tugas_modul_02_2310511001.c
```

2. Kartu Nilai Praktikum

Buat program C yang menerima nilai tugas dan nilai ujian seorang mahasiswa, lalu menghitung dan mencetak kartu nilai dengan format presisi seperti pada Contoh Output. Nilai akhir dihitung dengan bobot 40% tugas dan 60% ujian. Keluaran program dinilai secara ketat: harus sama persis karakter demi karakter, termasuk spasi.

Format Input

Dua baris angka bertipe double: nilai tugas dan nilai ujian.

Batasan

- $0 \leq$ nilai tugas ≤ 100 (dapat berupa bilangan pecahan);
- $0 \leq$ nilai ujian ≤ 100 (dapat berupa bilangan pecahan).

Contoh 1

Input:

```
80
90
```

Output:

```
====================================
            KARTU NILAI
====================================
Nilai Tugas  : 80.00
Nilai Ujian  : 90.00
------------------------------------
Nilai Akhir  : 86.00
====================================
```

Contoh 2

Input:

```
100
100
```

Output:

```
====================================
            KARTU NILAI
====================================
Nilai Tugas  : 100.00
Nilai Ujian  : 100.00
------------------------------------
Nilai Akhir  : 100.00
====================================
```

Contoh 3

Input:

```
60.5
```

```
75.25
```

Output:

```
====================================
            KARTU NILAI
====================================
Nilai Tugas  : 60.50
Nilai Ujian  : 75.25
------------------------------------
Nilai Akhir  : 69.35
====================================
```

Ketentuan

- gunakan variabel bertipe double untuk nilai_tugas, nilai_ujian, dan nilai_akhir;
- nilai_akhir dihitung dengan rumus (nilai_tugas * 0.4) + (nilai_ujian * 0.6);
- gunakan %.2f untuk seluruh nilai agar selalu tampil dua angka di belakang koma;
- kedua baris pembatas (= dan -) harus memiliki jumlah karakter yang sama seperti pada Contoh Output;
- keluaran diperiksa apa adanya (karakter demi karakter), sehingga kesalahan satu spasi tetap dianggap tidak sesuai.

Simpan sebagai tugas_modul_02_soal2_nim.c dan sertakan sebagai lampiran tambahan bersama tugas utama.

IX. Kesalahan Umum dan Standar Penulisan

1. Kesalahan yang Sering Terjadi

a. Tidak menggunakan & pada scanf()

Salah:

```
scanf("%d", umur);
```

Benar:

```
scanf("%d", &umur);
```

b. Format specifier tidak sesuai

Salah:

```
float nilai;

scanf("%d", &nilai);
```

Benar:

```
scanf("%f", &nilai);
```

c. Menggunakan %f untuk input double

Salah:

```
double harga;

scanf("%f", &harga);
```

Benar:

```
scanf("%lf", &harga);
```

d. Menggunakan %lf pada printf()

Untuk konsistensi dan kejelasan, tampilkan double dengan %f:

```
double harga = 12500.5;

printf("%.2f\n", harga);
```

e. Menggunakan petik ganda untuk char

Salah:

```
char kelas = "A";
```

Benar:

```
char kelas = 'A';
```

f. Menggunakan variabel sebelum diberi nilai

Tidak disarankan:

```
int jumlah;

printf("%d\n", jumlah);
```

Lebih baik:

```
int jumlah = 0;
```

atau menerima nilai melalui input sebelum digunakan.

g. Pembagian bilangan bulat

```
int hasil = 5 / 2;
```

Nilai hasil adalah:

```
2
```

Bukan:

```
2.5
```

Gunakan bilangan pecahan apabila menginginkan hasil pecahan:

```
double hasil = 5.0 / 2.0;
```

h. Tidak memeriksa warning compiler

Program dapat berhasil dikompilasi tetapi masih menghasilkan warning.

Warning tidak boleh langsung diabaikan. Warning dapat menunjukkan:

- format specifier yang tidak sesuai;
- variabel tidak digunakan;
- nilai belum diinisialisasi;
- konversi data yang berpotensi kehilangan informasi.

Aktifkan warning compiler apabila menggunakan GCC:

```
gcc -Wall -Wextra latihan_01.c -o latihan_01
```

2. Standar Penulisan Kode

Deklarasi variabel

Kelompokkan variabel secara logis:

```
double jarak;
double konsumsi;
double harga_bahan_bakar;
double kebutuhan_bahan_bakar;
double total_biaya;
double biaya_per_orang;
int jumlah_penumpang;
```

Nama deskriptif

Disarankan:

```
harga_bahan_bakar
```

Kurang informatif:

```
h
```

Spasi pada operator

Disarankan:

```
total_biaya = kebutuhan_bahan_bakar * harga_bahan_bakar;
```

Kurang rapi:

```
total_biaya=kebutuhan_bahan_bakar*harga_bahan_bakar;
```

Satu pernyataan per baris

Disarankan:

```
double panjang;
double lebar;
double luas;
```

Hindari penulisan yang terlalu padat apabila mengurangi keterbacaan.

X. Pernyataan penggunaan AI

Tidak menggunakan AI:

```
Saya menyatakan bahwa tugas ini dikerjakan tanpa menggunakan bantuan
AI generatif.
```

Menggunakan AI:

```
Saya menggunakan [nama perangkat AI] untuk membantu menjelaskan
[bagian yang dibantu]. Hasil tersebut saya periksa melalui
[kompilasi, pengujian, perhitungan manual, atau sumber pembelajaran].
```

Daftar Pustaka

Deitel, P., & Deitel, H. (2016). C: How to program (8th ed.). Pearson Education.

Free Software Foundation. (n.d.). GCC, the GNU Compiler Collection: Online documentation. GNU Project. Diakses dari https://gcc.gnu.org/onlinedocs/

International Organization for Standardization. (2018). ISO/IEC 9899:2018 — Information technology — Programming languages — C. ISO.

Kernighan, B. W., & Ritchie, D. M. (1988). The C programming language (2nd ed.). Prentice Hall.

Microsoft. (n.d.). C/C++ for Visual Studio Code. Visual Studio Code Documentation. Diakses dari https://code.visualstudio.com/docs/languages/cpp

Plauger, P. J. (1992). The standard C library. Prentice Hall.